

Design Proposal: Scaling JetStream Stream Sourcing

Status	Draft / Proposal
Area	JetStream — stream sourcing (<code>server/stream.go</code> , <code>server/consumer.go</code>)
Author	source-consumers-linear-scan investigation
Date	2026-06-06
Related	<code>jetstream_sourcing_scaling_test.go</code> , report <code>docs/reports/source-consumers-linear-scan.html</code>

1. Summary

JetStream stream **sourcing** scales poorly in the number of configured sources `S` . Several control-plane operations — consumer setup on leader election/restart, `STREAM.UPDATE` , and `STREAM.INFO` — are $O(S^2)$: they map a source *index name* (`iname`) back to its `*StreamSource` config through a linear walk (`stream.streamSource()` , `stream.go:3793`) that is itself called once per source.

The message **data path is already $O(1)$** and is unchanged by this proposal.

We propose two small derived indexes alongside `cfg.Sources` so that `iname` $\rightarrow$ `config` and `streamName` $\rightarrow$ `count` become $O(1)$. That collapses the affected operations from $O(S^2)$ to $O(S)$ with no behaviour change, no wire/disk/config change, and no migration.

2. First, what is an `iname` ?

An `iname` ("index name") is a **stable, unique string identifier for one source *within* a sourcing stream**. It is built by `composeIName()` (`stream.go:1079`) and used as the key of the

runtime map `mset.sources[iname]` and to correlate inbound messages/headers back to their source.

Anatomy of an iname (source index name)

A stable, unique string ID for one source inside a sourcing stream. Format: "idName source destination" (space-separated).

"ORDERS:7f3a9k2p orders.us >"



- ☐ origin stream name (here `ORDERS`)
- ☐ present only for `External` sources — 8-char `getHash(ApiPrefix)`, e.g. JS domain `$JS.edge1.API`
- ☐ (a durable consumer adds `:C=hash(consumerName)` too)
- ☐ filter subject, or `>` if none (transforms are joined with `\f`)
- ☐ destination transform, or `>` if none

Why it exists: one stream may source the same origin many times with different filters/domains, so `Name` alone is not unique — the iname is.

It is the concatenation of three space-separated parts — `idName source destination` :

Part	Comes from	Notes
<code>idName</code>	<code>Name</code> <code>[+ " : " +</code> <code>getHash(External.ApiPrefix)]</code> <code>[+ " :C="</code> <code>+ getHash(Consumer.Name)]</code>	the origin stream, plus an 8-char hash of the JS-domain API prefix for external sources, plus a durable-consumer hash if pinned
<code>source</code>	<code>FilterSubject</code> , or <code>></code> if empty	for multi-transform sources, the transform <i>sources</i> joined by <code>\f</code>
<code>destination</code>	transform destination, or <code>></code> if none	

Worked examples (`getHash` returns 8 base-36 chars; shown here as `7f3a9k2p`):

Source config	iname
<code>Name: "ORDERS"</code> , no filter, no transform, local	<code>ORDERS > ></code>
<code>Name: "ORDERS"</code> , <code>FilterSubject: "orders.us"</code>	<code>ORDERS orders.us ></code>
<code>Name: "ORDERS"</code> , <code>External.ApiPrefix: "\$JS.edge1.API"</code>	<code>ORDERS:7f3a9k2p > ></code>
same name on a <i>different</i> edge domain <code>\$JS.edge2.API</code>	<code>ORDERS:q1w2e3r4 > ></code>

Why it exists: one stream may source the *same* origin stream multiple times with different filters, transforms, or JS domains. `Name` alone is therefore not unique — the `iname` is.

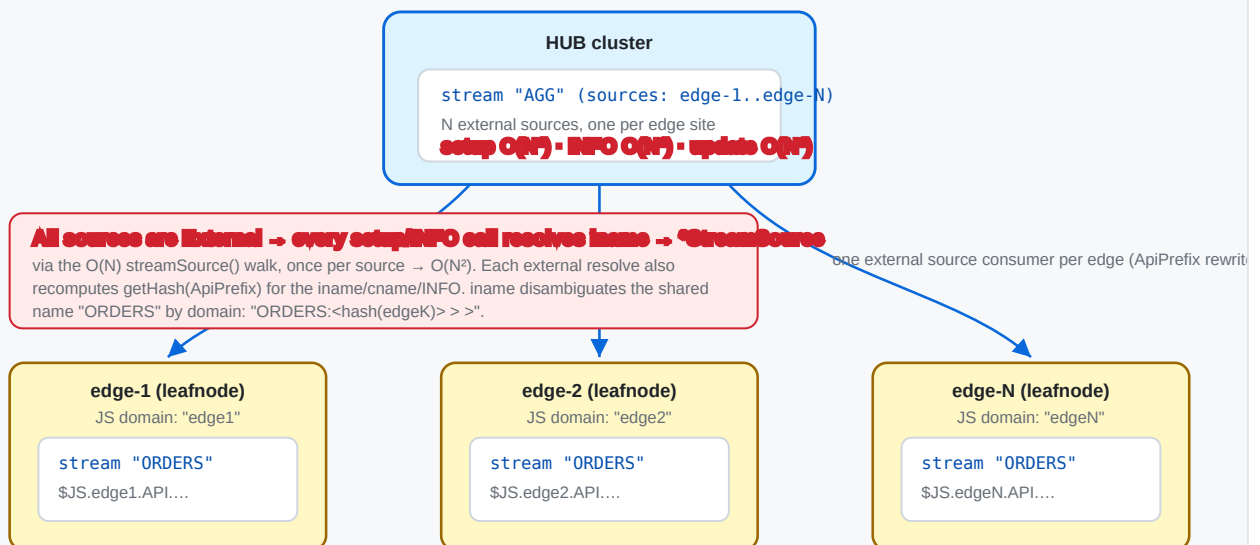
(The test at `norace_2_test.go:1016` looks up `mset.sources["TEST > >"]`, confirming the format.)

The whole performance problem is just this: the runtime state is **keyed by `iname` in a map**, but the *config* it points back to lives in a **slice with no `iname` key**.

3. Motivating scenario

A JetStream instance runs on an **edge leafnode site** with a stream in its own **JS domain**. From the **HUB** side we create a stream that **sources** from that edge leafnode stream — repeated across many edge sites.

Motivating case: HUB sources from N edge leafnode streams across JS domains



This is the worst case because **every** source is *external* (a per-domain `ApiPrefix` such as `$JS.edge1.API.>`), so every `setup/INFO/update` call has to resolve `iname` → `*StreamSource` to read that prefix — driving it straight through the $O(S^2)$ paths below. The shared name `ORDERS` across domains is fine for correctness (the domain hash in the `iname` disambiguates it), but it means the slice is full of look-alike entries that the linear walk cannot shortcut.

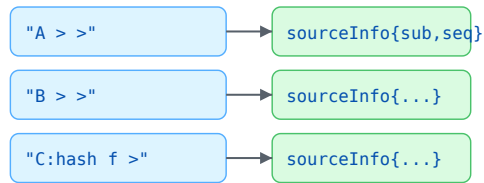
4. Root cause: a map on one side, a slice on the other

The asymmetry: runtime state is a map, config is a slice

mset.sources

map[string]*sourceInfo

keyed by iname — direct hashed lookup



~~mset.sources[iname] → O(1)~~

lookup cost independent of source count

mset.cfg.Sources

[]*StreamSource

positional only — no iname index



for _, ssi := range cfg.Sources { if ssi.iname == want

~~streamSource(iname) → O(1)~~

must walk every entry to map iname → config

stream.go:3793

```
// stream.go:515 — runtime state, keyed by iname → O(1)
sources map[string]*sourceInfo

// cfg.Sources — the CONFIG, a flat slice, no iname index → O(S) to find one
Sources []*StreamSource
```

So the single helper that maps `iname → config` must scan:

```
// stream.go:3793
func (mset *stream) streamSource(iname string) *StreamSource {
    for _, ssi := range mset.cfg.Sources { // O(S)
        if ssi.iname == iname {
            return ssi
        }
    }
    return nil
}
```

5. Why that is $O(S^2)$ — a worked example

`streamSource()` alone is $O(S)$. It becomes $O(S^2)$ because it is called **once per source from inside a loop that already visits every source**. Walk through `setupSourceConsumers` with `S = 5` (edge sources `E1..E5`):

- The outer loop iterates the slice in order: E1, E2, E3, E4, E5 .
- For each, it calls `setupSourceConsumer(iname)` → `trySetupSourceConsumer` → `streamSource(iname)` , which **re-walks the slice from the front** to find that same `iname` .
- E1 is found after 1 compare, E2 after 2, ... E5 after 5.

Why setup is $O(S^2)$: each of S setups re-scans the slice (example S = 5)

Rows = `setupSourceConsumers` iterating sources in order · Columns = `streamSource()` walking `cfg.Sources` to find that `iname`

	cmp[0]	cmp[1]	cmp[2]	cmp[3]	cmp[4]	
setup E1						1 compare
setup E2						2 compares
setup E3						3 compares
setup E4						4 compares
setup E5						5 compares

Total = 1+2+3+4+5 = 15 = 5*6/2

S = 5 → 15 · S = 100 → 5,050 · S = 1000 → 500,500 – grows as S^2 (red filled = wasted comparisons)

Total compares = $1 + 2 + 3 + 4 + 5 = 15 = S(S+1)/2$. That triangular sum is $O(S^2)$:

S	compares ($S(S+1)/2$)
5	15
100	5,050
1000	500,500

And `setupSourceConsumers` is only one of several $O(S^2)$ callers — combined, a single leader election on a 1000-source hub stream is on the order of **a million** string comparisons, all under `mset.mu` .

How one walk becomes $O(S^2)$: a loop inside a loop

`setupSourceConsumers()`

runs on leader election / restart / R1 start · stream.go:4805

$S \times O(S) = O(S^2)$

$S=1000 \rightarrow \sim 1,000,000$ string compares

for each of S sources $\rightarrow$ `setupSourceConsumer(iname)`

outer loop · S iterations

`trySetupSourceConsumer(iname)`

stream.go:3897 — needs the `*StreamSource` config for ext prefix, filters, start policy

`streamSource(iname) — for _, si := range cfg.Sources { ... }`

inner walk · up to S comparisons · stream.go:3793

`[0][1][2][3] ... [S]`

`[0][1][2][3] ... [S]`

× repeated for every

one of the S sources

All of this happens while holding `mset.mu`, blocking publish

What is *not* affected

The inbound data path stays $O(1)$: each source's delivery subscription closes over its own `si` (`stream.go:4161`) and passes it straight to `processInboundSourceMsg(si, ...)` (`stream.go:4314`). No lookup per message.

The data path is already $O(1)$ — only the control plane scans

DATA PATH — per inbound message — $O(1)$

`delivery sub closure`

binds its own `si`
stream.go:4161

`si (bound)`

`queueInbound(..., si)`

tags msg with `si`, single mux queue
processAllSourceMsgs · 4187

`processInboundSourceMsg(si)`

CONTROL PLANE — setup / election / update / INFO — scans `cfg.Sources`

`setupSourceConsumers` $O(S^2)$

`STREAM.UPDATE (sources)` $O(S^2)$

`STREAM.INFO → sourceInfo` $O(S^2)$

`startingSequenceForSources` $O(M \cdot S)$

all route through

`streamSource()` — $O(S)$ walk

6. The affected operations — before $\rightarrow$ after

Each subsection shows the operation, why it is quadratic today, and what it becomes with the proposed indexes (defined in §7).

6.1 setupSourceConsumers — leader election / restart / R1 start

Before — $O(S^2)$ (stream.go:4805 , 4811 , 3911)

```
for _, ssi := range mset.cfg.Sources {                // S iterations
    if si := mset.sources[ssi.iname]; si != nil {
        mset.setupSourceConsumer(ssi.iname, si.sseq+1, ...) // → trySetupSourceC
    }
}
// inside trySetupSourceConsumer (stream.go:3911):
ssi := mset.streamSource(iname)                       // O(S) walk, per source
```

Cost at $S=1000$: $\approx 500,500$ compares per election.

After — $O(S)$

```
// streamSource() is now a map read; the loop body is O(1).
ssi := mset.streamSource(iname) // mset.cfg.Sources[iname]
```

Cost at $S=1000$: 1000 map reads.

6.2 STREAM.INFO → sourcesInfo → sourceInfo — every info request

Before — $O(S^2)$ (stream.go:2953 , 2992)

```
func (mset *stream) sourcesInfo() (sis []*StreamSourceInfo) {
    for _, si := range mset.sources {                // S sources
        sis = append(sis, mset.sourceInfo(si))
    }
}
// sourceInfo, per source, for external sources:
} else if ss := mset.streamSource(si.iname); ss != nil && ss.External != nil { /
```

In the edge/hub case *all* sources are external, so every one pays the walk → $O(S^2)$ per INFO .
Monitoring that polls STREAM.INFO makes this continuous.

After — $O(S)$

```
} else if ss := mset.streamSource(si.iname); ss != nil && ss.External != nil { /
```

6.3 STREAM.UPDATE (sources) + createSourcingConsumerHash

Before — $O(S^2)$ twice over (stream.go:2509 , 2516 , 2523 ; consumer.go:6354)

The update diff derives a consumer name for every old and new source:

```
getSourcingConsumerIName := func(ssi *StreamSource, sources []*StreamSource) string {
    ...
    return fmt.Sprintf("%s %s", iName, mset.createSourcingConsumerHash(ssi, sources))
}
for _, s := range ocfg.Sources { ... getSourcingConsumerIName(s, ocfg.Sources) }
for _, s := range cfg.Sources { ... getSourcingConsumerIName(s, cfg.Sources) }
```

and createSourcingConsumerHash walks the slice to decide whether a stream name is used more than once:

```
// consumer.go:6354
var once bool
for _, src := range sources { // O(S)
    if src.Name == ssi.Name {
        if once { /* append iname, */ break } // duplicate name → breaks early
        once = true
    }
}
```

Subtlety worth getting right: this loop only **breaks early when the name is a duplicate**. For a name that is **unique** in the list (e.g. each edge stream named ORDERS-edge1 , ORDERS-edge2 , ...) there is no second match, so it scans the **entire slice** every call → $O(S)$ per call → $O(S^2)$ across the update. (When names collide it is cheap here, but you still pay §6.1/ §6.2.)

After — $O(S)$ using the precomputed name-count map:


```

if mset.cfgSourceNameCount[ssi.Name] > 1 { // 0(1), no scan, correct for unique
    if ssi.iname == _EMPTY_ { ssi.setIndexName() }
    id = fmt.Sprintf("%s %s", id, ssi.iname)
}

```

6.4 setStartingSequenceForSources — the standing // TODO

Before — up to $O(M \cdot S^2)$ (stream.go:4566 , 4638)

When resuming, this reverse-scans the store; for legacy (pre-2.10) source headers that carry only a stream name, it searches all sources and calls `streamSource()` **twice per candidate**:

```

for iName := range iNames {
    // TODO streamSource is a linear walk, to optimize later
    if si := mset.sources[iName]; si != nil && streamName == si.name ||
        (mset.streamSource(iName).External != nil && // wa
         streamName == si.name+": "+getHash(mset.streamSource(iName).External.Api
        ...
    }
}

```

With M messages scanned and S sources, the inner walks make this $O(M \cdot S^2)$ in the degenerate case.

After — $O(M \cdot S)$ (walks become map reads, removing the TODO):

```

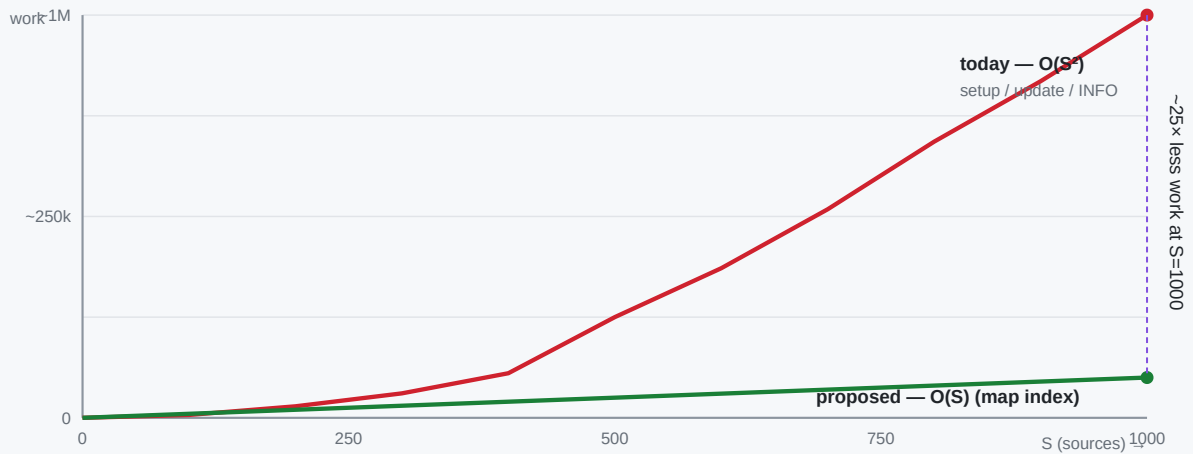
ssi := mset.streamSource(iName) // map read
if si := mset.sources[iName]; si != nil &&
    (streamName == si.name ||
     (ssi != nil && ssi.External != nil && streamName == si.name+": "+ssi.cachedApi
    ...
}

```

(`cachedApiHash` is the optional memoization from §7; even without it, the map read alone removes the $O(S^2)$ factor, leaving the unavoidable $O(M \cdot S)$ store scan.)

Summary of before → after

Work per control-plane event vs. number of sources (S)



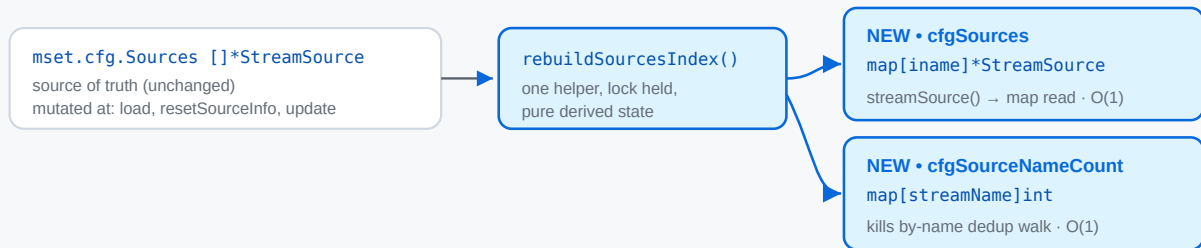
Operation	Trigger	Before	After
<code>setupSourceConsumers</code> (§6.1)	leader election / restart	$O(S^2)$	$O(S)$
<code>STREAM.INFO</code> / <code>sourcesInfo</code> (§6.2)	every info request	$O(S^2)$	$O(S)$
<code>STREAM.UPDATE</code> <code>sources</code> (§6.3)	config update	$O(S^2)$	$O(S)$
<code>createSourcingConsumerHash</code> (§6.3)	per source (unique names)	$O(S)$	$O(1)$
<code>setStartingSequenceForSources</code> (§6.4)	config update	$O(M \cdot S^2)$	$O(M \cdot S)$
<code>startingSequenceForSources</code> (legacy match)	setup / restart	$O(M \cdot S)$	$O(M)^*$
inbound message (data path)	per message	$O(1)$	$O(1)$ (unchanged)

* with a `map[streamName][]*StreamSource` for the legacy by-name match (optional, §7).

7. Proposal

Maintain derived indexes next to `cfg.Sources`, rebuilt under `mset.mu` whenever the list changes.

Proposal: derive two indexes when `cfg.Sources` changes



Effect on the control-plane callers

setupSourceConsumers	$O(N) \rightarrow O(N)$	STREAM.UPDATE (sources)	$O(N) \rightarrow O(N)$
sourceInfo / sourcesInfo (INFO)	$O(N) \rightarrow O(N)$	createSourcingConsumerHash	$O(N) \rightarrow O(1)$ dedup
setStartingSequenceForSources	drops the TODO walk	legacy header matching	map[name] lookup

```

// stream struct, near stream.go:515
sources          map[string]*sourceInfo // existing runtime state
cfgSources       map[string]*StreamSource // NEW: iname      -> config (Tier
cfgSourceNameCount map[string]int         // NEW: stream name -> count (Tier
  
```

```

// Lock held. Pure derived state – rebuilt on any cfg.Sources mutation.
func (mset *stream) rebuildSourcesIndex() {
    cs := make(map[string]*StreamSource, len(mset.cfg.Sources))
    nc := make(map[string]int, len(mset.cfg.Sources))
    for _, ssi := range mset.cfg.Sources {
        if ssi.iname == _EMPTY_ {
            ssi.setIndexName()
        }
        cs[ssi.iname] = ssi
        nc[ssi.Name]++
    }
    mset.cfgSources, mset.cfgSourceNameCount = cs, nc
}

// O(1) replacement for the linear walk.
func (mset *stream) streamSource(iname string) *StreamSource {
    return mset.cfgSources[iname]
}
  
```

Rebuild call sites (the only places `cfg.Sources` changes): config load / `resetSourceInfo` (`stream.go:4659`) and the `STREAM.UPDATE` source diff where sources are appended/removed

(stream.go:2530 , 2570).

Tiers

- **Tier 1 — cfgSources index.** Fixes §6.1, §6.2, §6.4 by turning streamSource() into a map read. Smallest, highest-impact change.
 - **Tier 2 — cfgSourceNameCount .** Fixes §6.3 dedup; optionally add map[streamName][]*StreamSource for the legacy by-name match, and memoize getHash(ApiPrefix) per source (cachedApiHash) so external iname / cname /INFO construction stops re-hashing.
 - **Tier 3 — validation.** BenchmarkSetupSourceConsumers over $S \in \{100, 500, 1000\}$ plus a down-scaled, *un-skipped* external/domain variant of TestStreamSourcingScalingSourcingManyBenchmark (jetstream_sourcing_scaling_test.go:110). Run -race .
-

8. Goals / Non-goals

Goals: make the source control plane $O(S)$; behaviour-preserving (no wire/disk/config change); specifically kill the cost in the edge → hub external case.

Non-goals: changing data-path throughput or sharding the single source goroutine (TODO(dlc) , stream.go:4187); changing iname / cname composition or the source header format.

9. Alternatives considered

- **Sort cfg.Sources + binary search** — $O(\log S)$, and re-sorting complicates the append-based update path; a map is simpler and $O(1)$.
 - **Store *StreamSource on sourceInfo** — some callers only hold the config slice; a standalone index serves all callers uniformly and keeps the structs decoupled.
 - **Document only / do nothing** — the edge → hub fan-in is a real, growing topology; $O(S^2)$ degrades leader elections and INFO polls as sites are added.
-

10. Risks & compatibility

- **Index drift** — must rebuild on every `cfg.Sources` mutation. Mitigation: funnel mutations through `rebuildSourcesIndex()` ; assert `len(cfgSources) == len(cfg.Sources)` in test builds.
- **Uniqueness** — `iname` is already validated unique per stream (`stream.go:1963-1974`), so the key is well-defined.
- **Concurrency** — indexes read/written only under `mset.mu` , exactly like `mset.sources` .
- **Wire/disk/config** — unchanged; pure in-memory performance change.

11. Rollout

1. Tier 1 (`cfgSources`) as a self-contained, easy-to-review PR + benchmark.
2. Tier 2 (`cfgSourceNameCount` + hash memoization) as a follow-up, motivated by the edge/hub benchmark.
3. Keep the existing scaling test as the acceptance gate; add the external/domain variant.

Appendix — key references

Symbol	File:line	Role
<code>composeIName</code> / <code>composeCName</code>	<code>stream.go:1079</code> / <code>1129</code>	builds the <code>iname</code> (incl. <code>getHash(ApiPrefix)</code>)
<code>streamSource</code>	<code>stream.go:3793</code>	the linear scan (Tier 1 target)
<code>sources</code> map / <code>cfg.Sources</code> slice	<code>stream.go:515</code>	the asymmetry
<code>setupSourceConsumers</code>	<code>stream.go:4805</code>	§6.1 $O(S^2)$ setup loop
<code>trySetupSourceConsumer</code>	<code>stream.go:3897</code>	external <code>ApiPrefix</code> rewrite; per-source <code>streamSource</code>
<code>sourceInfo</code> / <code>sourcesInfo</code>	<code>stream.go:2964</code> / <code>2953</code>	§6.2 INFO $O(S^2)$
update sources diff	<code>stream.go:2504</code>	§6.3 update path
<code>createSourcingConsumerHash</code>	<code>consumer.go:6354</code>	§6.3 by-name dedup (Tier 2 target)
<code>setStartingSequenceForSources</code>	<code>stream.go:4566</code>	§6.4 TODO-flagged $O(S^2)$

Symbol	File:line	Role
startingSequenceForSources	stream.go:4694	reverse-scan resume
processInboundSourceMsg	stream.go:4314	O(1) hot path (unchanged)
getHash	events.go:1157	8-char hash used in <code>iname</code>